

DCI-VTON

AWS GPU Deployment Guide

Virtual Try-On System - Production Deployment

Version 1.0 | June 2026

Architecture: FastAPI + Celery + GPU Inference (T4/A10G)

Target Inference Time: < 30 seconds per job

GPU VRAM Required: ~5GB (T4 16GB recommended)

Table of Contents

1.	Architecture Overview	3
2.	Prerequisites & AWS Account Setup	4
3.	GPU Instance Selection & Launch	5
4.	Server Environment Setup	7
5.	Model Weights Download	8
6.	Project Code Deployment	9
7.	Environment Configuration	10
8.	Docker Build & Launch	11
9.	Testing & Verification	12
10.	Domain & SSL Setup	13
11.	Cost Optimization	14
12.	Monitoring & Maintenance	15
13.	Troubleshooting	16

1. Architecture Overview

1.1 System Architecture

The DCI-VTON system consists of the following components running on a single AWS GPU instance:

- Frontend (Next.js) - Port 3000 - User uploads person + garment images
- API Server (FastAPI) - Port 8000 - Handles requests, stores images
- Task Queue (Celery + Redis) - Async job processing
- GPU Worker (Celery) - Runs DCI-VTON inference on GPU
- Database (PostgreSQL) - Job status tracking
- Flower (Optional) - Port 5555 - Celery monitoring dashboard

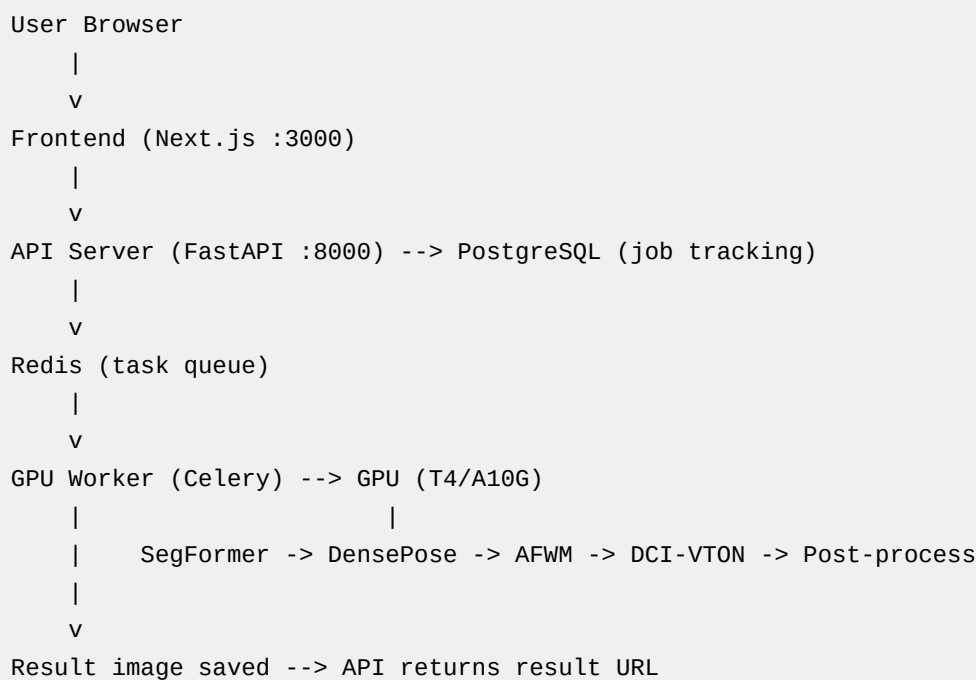
1.2 Inference Pipeline (per job)

Each try-on job executes these ML models sequentially on the GPU:

- SegFormer - Clothing segmentation (~500MB, ~2 sec)
- DensePose - Body pose estimation (~250MB, ~3 sec)
- AFWM - Garment warping to body shape (~200MB, ~2 sec)
- DCI-VTON - Diffusion-based try-on generation (~3.5GB, ~15 sec)
- Post-processing - Color correction + compositing (~1 sec)

Total: ~4.5GB VRAM, ~20-30 seconds per job. All models are preloaded at worker startup (one-time ~3 min), then each job only runs inference.

1.3 Request Flow



2. Prerequisites & AWS Account Setup

2.1 AWS Account

- Active AWS account with billing enabled
- IAM user with AdministratorAccess (or EC2 + VPC + Route53 permissions)
- AWS CLI v2 installed locally: <https://aws.amazon.com/cli/>

2.2 Configure AWS CLI

```
aws configure
AWS Access Key ID: <your-access-key>
AWS Secret Access Key: <your-secret-key>
Default region: ap-south-1 (or your preferred region)
Default output format: json
```

2.3 Request GPU Quota (CRITICAL - Do this FIRST)

WARNING: GPU quota request can take 1-24 hours. Submit immediately before proceeding.

AWS blocks GPU instance launches by default. You must request a quota increase:

1. Go to: AWS Console > Service Quotas > EC2
2. Search: "Running On-Demand G and VT instances"
3. Click "Request increase at account level"
4. New quota value: 4 (g4dn.xlarge needs 4 vCPUs)
5. Use case: "ML inference for virtual try-on application"
6. Submit and wait for approval (usually 1-24 hours)

2.4 Set Billing Alert

1. AWS Console > Billing > Budgets > Create Budget
2. Budget type: Cost budget
3. Budget amount: \$100/month
4. Alert threshold: 80%
5. Email notification: your-email@example.com

2.5 Tools Required on Local Machine

- Git (for cloning repository)
- SSH client (built-in on Mac/Linux, PuTTY on Windows)
- SCP or rsync (for file upload to server)

3. GPU Instance Selection & Launch

3.1 Recommended Instance Types

Choose based on your budget and performance needs:

Instance	GPU	VRAM	vCPU	RAM	Cost/hr	Cost/mo(24x7)
-----	-----	-----	-----	-----	-----	-----
g4dn.xlarge	T4	16GB	4	16GB	\$0.526	~\$380
g5.xlarge	A10G	24GB	4	16GB	\$1.006	~\$725
g4dn.2xlarge	T4	16GB	8	32GB	\$0.752	~\$540

RECOMMENDED: g4dn.xlarge (same T4 GPU as Kaggle, cheapest option)

Our model needs ~5GB VRAM - T4's 16GB is more than enough.

3.2 Create Key Pair

```
aws ec2 create-key-pair \
  --key-name vton-key \
  --query 'KeyMaterial' \
  --output text > vton-key.pem

chmod 400 vton-key.pem    # Linux/Mac
# Windows: Properties > Security > Remove all except your user
```

3.3 Create Security Group

```
# Create security group
aws ec2 create-security-group \
  --group-name vton-sg \
  --description "VTON application security group"

# Allow SSH (restrict to your IP for security)
aws ec2 authorize-security-group-ingress \
  --group-name vton-sg --protocol tcp --port 22 \
  --cidr $(curl -s ifconfig.me)/32

# Allow HTTP, HTTPS, API, Frontend
aws ec2 authorize-security-group-ingress \
  --group-name vton-sg --protocol tcp --port 80 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress \
  --group-name vton-sg --protocol tcp --port 443 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress \
  --group-name vton-sg --protocol tcp --port 8000 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress \
  --group-name vton-sg --protocol tcp --port 3000 --cidr 0.0.0.0/0
```

3.4 Launch EC2 Instance

WARNING: Use Deep Learning AMI - it has CUDA, PyTorch, Docker pre-installed.

```
# Find latest Deep Learning AMI ID for your region
aws ec2 describe-images \
  --owners amazon \
  --filters "Name=name,Values=Deep Learning Base OSS Nvidia Driver GPU AMI*" \
    "Name=architecture,Values=x86_64" \
  --query 'Images | sort_by(@,&CreationDate) | [-1].ImageId' \
  --output text

# Launch instance (replace AMI_ID with result above)
aws ec2 run-instances \
  --image-id <AMI_ID> \
  --instance-type g4dn.xlarge \
  --key-name vton-key \
  --security-groups vton-sg \
  --block-device-mappings '[{
    "DeviceName": "/dev/sda1",
    "Ebs": {
      "VolumeSize": 150,
      "VolumeType": "gp3",
      "Iops": 3000,
      "Throughput": 125
    }
  }]' \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=vton-gpu}]'
```

3.5 Allocate Elastic IP (Fixed IP Address)

```
# Allocate
aws ec2 allocate-address --domain vpc

# Note the AllocationId from output, then associate:
aws ec2 associate-address \
    --instance-id <INSTANCE_ID> \
    --allocation-id <ALLOCATION_ID>

# Note the PUBLIC IP - this is your server's permanent address
```

4. Server Environment Setup

4.1 SSH into Server

```
ssh -i vton-key.pem ubuntu@<ELASTIC_IP>
```

4.2 Verify GPU

```
nvidia-smi

# Expected output should show:
# Tesla T4, 16GB VRAM, CUDA 12.x
# If GPU not visible, the AMI may need nvidia driver setup
```

4.3 Install NVIDIA Container Toolkit (Docker GPU Support)

```
# Add NVIDIA repo
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
    sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/$distribution/libnvidia-container.list | \
    sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https:'>
    sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

# Install
sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit

# Configure Docker
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker

# Verify GPU in Docker
docker run --rm --gpus all nvidia/cuda:12.1.0-base-ubuntu22.04 nvidia-smi
```

4.4 Install Docker Compose v2

```
# Docker Compose plugin (v2)
sudo apt-get install docker-compose-plugin

# Verify
docker compose version
```


5. Model Weights Download

WARNING: Model weights are ~4GB total. Download directly on the server (faster than upload).

5.1 Create Weights Directory

```
mkdir -p /home/ubuntu/vton_project/ml/weights
cd /home/ubuntu/vton_project/ml/weights
```

5.2 Download DCI-VTON Checkpoint (viton512.ckpt)

This is the main diffusion model (~3.5GB). Download from the Kaggle dataset:

```
# Option A: Using Kaggle CLI
pip install kaggle
export KAGGLE_USERNAME=aisubscrip2
export KAGGLE_KEY=<your-kaggle-key>

kaggle datasets download aisubscrip2/dci-vton-weights \
  --file viton512.ckpt \
  -p /home/ubuntu/vton_project/ml/weights/ --force

# Unzip if downloaded as .zip
cd /home/ubuntu/vton_project/ml/weights/
unzip -o *.zip 2>/dev/null; rm -f *.zip

# Option B: Using direct URL (if hosted on S3/GDrive)
# wget <direct-url> -O viton512.ckpt
```

5.3 Download AFWM Warp Weights (warp_viton.pth)

```
kaggle datasets download aisubscrip2/dci-vton-weights \
  --file warp_viton.pth \
  -p /home/ubuntu/vton_project/ml/weights/ --force

unzip -o *.zip 2>/dev/null; rm -f *.zip
```

5.4 DensePose Weights (auto-downloaded)

DensePose weights (~250MB) are automatically downloaded on first run. No manual action needed. The file will be saved to: /home/ubuntu/vton_project/ml/weights/densepose_rcnn_R_50_FPN_s1x.pkl

5.5 Verify All Weights

```
ls -lh /home/ubuntu/vton_project/ml/weights/
```

```
# Expected:
```

```
# viton512.ckpt          ~3.5GB  (DCI-VTON diffusion model)
```

```
# warp_viton.pth         ~200MB  (AFWM garment warp model)
```

```
# densepose_*.pkl       ~250MB  (auto-downloaded on first run)
```

```
# CRITICAL: Both viton512.ckpt and warp_viton.pth MUST exist
```

```
# If missing, the worker will fail to start
```

6. Project Code Deployment

6.1 Clone Repository

```
cd /home/ubuntu
git clone <your-repo-url> vton_project
cd vton_project
```

6.2 Project Structure (Key Files)

```
vton_project/
  api/
    Dockerfile.gpu           # GPU Docker image
    docker-compose.gpu.yml  # GPU docker-compose
    requirements.txt         # Base Python deps
    requirements-gpu.txt     # GPU Python deps (PyTorch, detectron2, etc.)
  app/
    main.py                 # FastAPI app
    config.py               # Settings (reads .env)
    services/
      inference.py          # Inference router (auto-detects GPU mode)
    workers/
      tasks.py              # Celery task definitions
  ml/
    weights/                # Model weights (Step 5)
      viton512.ckpt
      warp_viton.pth
    scripts/
      gpu_inference.py      # GPU inference engine
    kaggle/
      dci_vton_inference.ipynb # Kaggle notebook (not used in GPU mode)
  storage/                 # Runtime storage (auto-created)
    inputs/                 # Uploaded images
    outputs/                # Result images
  frontend/                # Next.js frontend
  .env                     # Environment configuration
```

6.3 Verify Weights are in Correct Location

```
# Weights must be at: vton_project/ml/weights/
ls -lh /home/ubuntu/vton_project/ml/weights/viton512.ckpt
ls -lh /home/ubuntu/vton_project/ml/weights/warp_viton.pth

# If weights were downloaded elsewhere, move them:
mv /path/to/viton512.ckpt /home/ubuntu/vton_project/ml/weights/
mv /path/to/warp_viton.pth /home/ubuntu/vton_project/ml/weights/
```

7. Environment Configuration

7.1 Edit .env File

```
cd /home/ubuntu/vton_project
nano .env
```

7.2 Required .env Settings for GPU Mode

```
# ---- Database ----
DATABASE_URL=postgresql+asyncpg://vton:vton@postgres:5432/vton

# ---- Celery / Redis ----
CELERY_BROKER_URL=redis://redis:6379/0
CELERY_RESULT_BACKEND=redis://redis:6379/0

# ---- Storage ----
STORAGE_BACKEND=local
LOCAL_STORAGE_PATH=/app/storage

# ---- ML / Inference (GPU MODE) ----
USE_OWN_MODEL=false
MODEL_PATH=
DEVICE=cuda
WEIGHTS_DIR=/app/ml/weights

# ---- Quality threshold ----
MIN_QUALITY_SCORE=0.65
TRAINING_PAIR_SSIM_THRESHOLD=0.65

# ---- API ----
MAX_UPLOAD_SIZE_MB=10

# ---- Kaggle (NOT NEEDED for GPU mode, can leave empty) ----
KAGGLE_USERNAME=
KAGGLE_KEY=
KAGGLE_NOTEBOOK_SLUG=
KAGGLE_DATASET_SLUG=
```

WARNING: DEVICE=cuda and WEIGHTS_DIR=/app/ml/weights are CRITICAL. Without these, GPU mode won't activate.

7.3 How Mode Detection Works

The inference router auto-detects the mode based on .env settings:

Priority order:

1. DEVICE=cuda + WEIGHTS_DIR set + weights exist --> Local GPU mode
2. KAGGLE_USERNAME + KAGGLE_KEY set --> Kaggle mode
3. Neither --> Placeholder mode

GPU mode: ~20-30 sec per job (direct GPU inference)

Kaggle mode: ~5-10 min per job (remote notebook execution)

Placeholder mode: ~3 sec (fake result for testing)

8. Docker Build & Launch

8.1 Create Docker Network

```
docker network create vton_net 2>/dev/null || true
```

8.2 Build and Start All Services

```
cd /home/ubuntu/vton_project/api

# Build (first time: ~15-20 min for GPU dependencies)
docker compose -f docker-compose.gpu.yml build

# Start all services
docker compose -f docker-compose.gpu.yml up -d

# Check all containers are running
docker compose -f docker-compose.gpu.yml ps
```

8.3 Expected Output

NAME	STATUS	PORTS
api-api-1	Up	0.0.0.0:8000->8000/tcp
api-worker-1	Up	(GPU attached)
api-postgres-1	Up (healthy)	0.0.0.0:5433->5432/tcp
api-redis-1	Up (healthy)	0.0.0.0:6379->6379/tcp
api-flower-1	Up	0.0.0.0:5555->5555/tcp

8.4 Check Worker GPU Access

```
# Verify GPU is accessible inside worker container
docker exec api-worker-1 nvidia-smi

# Check worker logs - should show "Local GPU mode active"
docker logs api-worker-1 --tail 30

# Expected log lines:
# InferenceRouter: Local GPU mode - loading models...
# SegFormer loaded.
# DensePose loaded.
# AFWM warp loaded.
# DCI-VTON model loaded.
# GPUInferenceEngine: all models loaded.
# InferenceRouter: Local GPU mode active.
```

8.5 First Run Model Loading

On first worker start, models take ~3-5 minutes to load into GPU memory. Subsequent job requests will be fast (~20-30

sec). DensePose weights will auto-download on first run (~250MB).

9. Testing & Verification

9.1 Health Check

```
curl http://localhost:8000/docs
# Should return FastAPI Swagger UI HTML
```

9.2 Submit Test Job via curl

```
# Upload person + garment images and submit try-on job
curl -X POST http://localhost:8000/api/tryon \
  -F "person_image=@/path/to/test_person.jpg" \
  -F "garment_image=@/path/to/test_garment.jpg"

# Response:
# {"job_id": "abc-123-def", "status": "pending", "eta_seconds": 35}
```

9.3 Check Job Status

```
curl http://localhost:8000/api/tryon/<job_id>

# Wait 20-30 seconds, then check again
# When status="completed", result_image_path will have the output
```

9.4 Check Worker Logs

```
docker logs api-worker-1 --tail 50 -f

# Watch for:
# [INFO] SegFormer loaded
# [INFO] DCI inference complete!
# [INFO] Result saved: /app/storage/outputs/abc-123.jpg
```

9.5 Performance Verification


```
# Expected timings per job:
# SegFormer segmentation:    ~2 sec
# DensePose extraction:      ~3 sec
# AFWM garment warping:      ~2 sec
# DCI-VTON diffusion:        ~15 sec
# Post-processing:           ~1 sec
# TOTAL:                     ~23 sec

# If inference takes >60 sec, check:
# 1. nvidia-smi (GPU utilization should be >80%)
# 2. docker stats (memory usage)
# 3. Worker logs for errors
```

10. Domain & SSL Setup (Optional)

10.1 Install Nginx

```
sudo apt-get update
sudo apt-get install -y nginx
```

10.2 Configure Nginx

```
sudo nano /etc/nginx/sites-available/vton

# Paste this configuration:
server {
    listen 80;
    server_name api.yourdomain.com;

    client_max_body_size 20M;

    location / {
        proxy_pass http://localhost:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 300s;
    }
}

server {
    listen 80;
    server_name app.yourdomain.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

# Enable site
sudo ln -s /etc/nginx/sites-available/vton /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

10.3 SSL with Let's Encrypt

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d api.yourdomain.com -d app.yourdomain.com

# Auto-renewal test
sudo certbot renew --dry-run
```

11. Cost Optimization

11.1 Cost Comparison

Option	Cost/hour	Cost/month(24x7)	Best for
----	-----	-----	-----
On-Demand	\$0.526	~\$380	Always-on production
Spot Instance	\$0.158	~\$115	Dev/testing (can interrupt)
Reserved 1-year	\$0.330	~\$240	Committed production
Reserved 3-year	\$0.220	~\$160	Long-term production
Auto Start/Stop	varies	~\$50-100	Low traffic

11.2 Auto Start/Stop (Recommended for Low Traffic)

Create a Lambda function that stops the instance when no API requests for 30 minutes, and a simple health-check endpoint that starts the instance on demand.

11.3 Spot Instance (70% Cheaper)

```
aws ec2 request-spot-instances \
  --instance-count 1 \
  --type persistent \
  --launch-specification '{
    "ImageId": "<AMI_ID>",
    "InstanceType": "g4dn.xlarge",
    "KeyName": "vton-key",
    "SecurityGroups": ["vton-sg"]
  }'

# WARNING: AWS can terminate spot instances with 2-min notice
# Use for development/testing only, not production
```

12. Monitoring & Maintenance

12.1 Docker Container Monitoring

```
# View all container status
docker compose -f docker-compose.gpu.yml ps

# View resource usage
docker stats

# View worker logs (live)
docker logs api-worker-1 -f --tail 100

# View API logs
docker logs api-api-1 -f --tail 100
```

12.2 GPU Monitoring

```
# Real-time GPU usage
watch -n 1 nvidia-smi

# Inside worker container
docker exec api-worker-1 nvidia-smi
```

12.3 Flower Dashboard (Celery Monitoring)

Access at http://<ELASTIC_IP>:5555 - shows active tasks, worker status, and task history.

12.4 Restart Services

```
# Restart specific service
docker compose -f docker-compose.gpu.yml restart worker

# Restart all
docker compose -f docker-compose.gpu.yml restart

# Full rebuild (after code changes)
docker compose -f docker-compose.gpu.yml up --build -d
```

12.5 Auto-Recovery

```
# Docker restart policy is set in docker-compose.gpu.yml
# Add to worker service if not present:
#   restart: unless-stopped

# EC2 auto-recovery:
aws ec2 modify-instance-attribute \
  --instance-id <INSTANCE_ID> \
  --auto-recovery-enabled
```

13. Troubleshooting

Problem: Worker fails to start

```
# Check logs
docker logs api-worker-1

# Common causes:
# 1. Weights missing: "FileNotFoundError: viton512.ckpt"
#   Fix: Download weights to ml/weights/ (see Step 5)
#
# 2. GPU not accessible: "CUDA not available"
#   Fix: Verify nvidia-container-toolkit installed (see Step 4.3)
#
# 3. Out of VRAM: "CUDA out of memory"
#   Fix: Use g4dn.xlarge (16GB) or reduce batch size
```

Problem: Inference takes >60 seconds

```
# Check GPU utilization
docker exec api-worker-1 nvidia-smi

# If GPU util = 0%: model not on GPU
#   Check DEVICE=cuda in .env
#
# If GPU util = 100% but slow: thermal throttling
#   Check: nvidia-smi -q -d TEMPERATURE
#
# If GPU util normal but slow: CPU bottleneck
#   Check: docker stats (CPU usage)
```

Problem: CUDA version mismatch

```
# Inside container:
docker exec api-worker-1 python -c "import torch; print(torch.version.cuda)"

# Must match the NVIDIA driver on host
nvidia-smi # Shows driver CUDA version

# Fix: Use compatible PyTorch version in Dockerfile.gpu
```

Problem: Port already in use

```
# Find what's using the port
sudo lsof -i :8000
sudo lsof -i :5432

# Kill the process or change port in docker-compose.gpu.yml
```

Problem: Database connection refused

```
# Check PostgreSQL is healthy
docker compose -f docker-compose.gpu.yml ps postgres

# If unhealthy, check logs
docker logs api-postgres-1

# Reset database (WARNING: deletes all data)
docker compose -f docker-compose.gpu.yml down -v
docker compose -f docker-compose.gpu.yml up -d
```


Quick Reference Card

Key Commands

```
# SSH into server
ssh -i vton-key.pem ubuntu@<ELASTIC_IP>

# Start services
cd /home/ubuntu/vton_project/api
docker compose -f docker-compose.gpu.yml up -d

# Stop services
docker compose -f docker-compose.gpu.yml down

# View logs
docker logs api-worker-1 -f

# Rebuild after code change
docker compose -f docker-compose.gpu.yml up --build -d

# Check GPU
nvidia-smi
docker exec api-worker-1 nvidia-smi

# Test API
curl -X POST http://<IP>:8000/api/tryon \
  -F person_image=@person.jpg \
  -F garment_image=@garment.jpg
```

Key URLs

```
API:      http://<ELASTIC_IP>:8000
API Docs: http://<ELASTIC_IP>:8000/docs
Frontend: http://<ELASTIC_IP>:3000
Flower:   http://<ELASTIC_IP>:5555
```

Key Files on Server

/home/ubuntu/vton_project/	
.env	# Configuration
api/docker-compose.gpu.yml	# GPU Docker compose
ml/weights/viton512.ckpt	# DCI-VTON model (~3.5GB)
ml/weights/warp_viton.pth	# AFWM warp model (~200MB)
storage/outputs/	# Generated results